UNIVERSITEIT VAN PRETORIA
Department of Computer Science

COS314 – Artificial Intelligence

Assignment 2 Investigation Report

Genetic Algorithm vs Iterated Local Search for

Due Date: 18 April 2026

Team Members: Mihlali Nondlozi: u26815983
Martin Phalane : u26535272

1. Background

The **0/1 Knapsack Problem** is one of the most extensively studied combinatorial optimisation problems in computer science. Given a set of *n* items each with a weight w_i and a value v_i, and a knapsack of maximum weight capacity *W*, the objective is to select a subset of items that maximises the total value while not exceeding the weight constraint (GeeksforGeeks, 2023). Put simply, one has to maximize the benefit of objects in a knapsack without exceeding its capacity:

$$\text{Maximise} \quad \Sigma\, v_i\, x_i \quad \text{subject to} \quad \Sigma\, w_i\, x_i \leq W, \quad x_i \in \{0, 1\}$$

The problem is classified as NP-hard (Karp, 1972), meaning no known polynomial-time exact algorithm exists for its general form. As Russell and Norvig (2022, Chapter 4) discuss in *Artificial Intelligence: A Modern Approach*, such problems require heuristic or metaheuristic approaches for practical-sized instances. Metaheuristics are broadly classified into two categories: **population-based** methods, which maintain and evolve a set of solutions, and **trajectory-based** methods, which follow a single solution path through the search space. This investigation compares one algorithm from each category where Mihlali worked on the GA and Martin ILS.

2. Genetic Algorithm (GA)

2.1 Description and Theoretical Foundation

The Genetic Algorithm is a **population-based metaheuristic** inspired by the Darwinian theory of natural selection and genetic inheritance. It is a search heuristic algorithm that mimics the cognitive process of natural selection, in which the best individuals are selected for social reproduction in order to create children or progeny for the following generation. Using the natural selection theory, this technique can be used to solve both confined and unconstrained problems. The genetic algorithm is based on biological mechanics (Shahbaz, 2020).Russell and Norvig (2022, p. 151) describe GAs as combining an upward step analogous to selection with random exploration through crossover and mutation. The core idea is that a population of candidate solutions (*chromosomes*) evolves over successive generations, with fitter individuals producing more offspring.

In the context of the 0/1 Knapsack Problem, each chromosome is a binary array of length *n*, where bit *i* = 1 indicates item *i* is included in the knapsack. The fitness of a chromosome is the total value of selected items if the total weight is within capacity, or zero otherwise (infeasible solutions are repaired, never directly penalised).

2.2 Algorithm Steps

- **Initialisation:** 100 chromosomes is created randomly with each gene set to 1 with probability 0.5.
- **Evaluation:** Each chromosome is evaluated using the fitness function. Infeasible solutions are repaired immediately. Select the best, discard the rest.
- **Selection:** Two individuals are drawn randomly and the fitter one is selected as a parent. This applies selection pressure while maintaining diversity.

- **Single-Point Crossover (rate 0.85):** A random cut-point is chosen; child 1 takes genes [0..cut] from parent 1 and [cut+1..n] from parent 2, child 2 takes the complement.
- **Bit-Flip Mutation (rate 0.02 per gene):** Each gene is independently flipped with probability 0.02. This introduces new genetic material and prevents premature convergence.
- **Repair Operator:** If a chromosome exceeds capacity W, items are removed in ascending value/weight ratio order until feasibility is restored. This greedy repair preserves as much value as possible.
- **Elitism:** The best individual from each generation is always copied unchanged into the next generation, ensuring the best solution is never lost.
- **Termination:** The algorithm halts after 500 generations.

2.3 Configuration

Parameter	Value
Population Size	100
Max Generations	500
Crossover Rate	0.85 (85%)
Mutation Rate	0.02 (2% per gene)
Selection Strategy	Binary Tournament (k = 2)
Crossover Type	Single-Point
Mutation Type	Bit-Flip
Elitism	Top-1 individual preserved
Repair Strategy	Greedy removal by lowest value/weight ratio
Termination	500 generations

2.4 Hypothesis

Based on the GA's ability to explore a large solution space through crossover (which combines good partial solutions from multiple parents) and maintain population diversity through mutation, we hypothesise that the GA will **perform competitively across all instances** and may find globally better solutions than ILS on harder, larger instances (e.g., f8 with 23 items). However, the GA is expected to have **significantly higher runtime** than ILS due to the overhead of evaluating an entire population per generation. On small instances (f3, f4, f7), both algorithms are expected to find the global optimum trivially.

3. Iterated Local Search (ILS)

3.1 Description and Theoretical Foundation

Iterated Local Search is a **trajectory-based metaheuristic** that extends simple hill-climbing by repeatedly escaping local optima through a perturbation step. Russell and Norvig (2022, p. 129) describe local search as algorithms that operate on a single current state and move to a neighbouring state. ILS builds on this by embedding local search inside an outer loop that uses intelligent restarts to explore different regions of the search space.

The key insight of ILS, as described by Lourenço, Martin and Stützle (2019), is that the **sequence of local optima** visited is itself a trajectory through a higher-level search space. By perturbing the current best solution and re-applying local search, ILS navigates this higher-level landscape, typically finding better solutions than random restarts.

3.2 Algorithm Steps

- **Greedy Initialisation:** Items are sorted by descending value/weight ratio and added greedily as long as they fit. This constructs a strong initial solution, giving ILS a better starting local optimum than a random start would.
- **Local Search – Best-Improvement 1-Bit Flip:** All n neighbouring solutions (obtained by flipping exactly one bit) are evaluated exhaustively. The best improving neighbour is accepted. This repeats until no single flip improves the objective (local optimum reached).
- **Perturbation – k-Bit Flip (k=4):** Four randomly chosen bits are flipped simultaneously. This is equivalent to the 'double-bridge' move used in TSP-ILS it creates a jump large enough to escape the current basin of attraction while retaining most of the solution structure.
- **Repair:** The perturbed solution is repaired using the same greedy repair as the GA.
- **Acceptance – Strict Improvement:** The perturbed local optimum replaces the incumbent only if it strictly improves the objective value. This is a greedy acceptance criterion.
- **Termination:** The algorithm halts after 1000 ILS iterations.

3.3 Configuration

Parameter	Value
Max Iterations	1000
Perturbation Strength	4 (bits flipped)
Initial Solution	Greedy (descending v/w ratio)
Local Search	Best-Improvement 1-Bit Flip
Acceptance Criterion	Strict Improvement
Repair Strategy	Greedy removal by lowest value/weight ratio
Termination	1000 iterations without global improvement

3.4 Hypothesis

ILS is expected to **execute significantly faster** than the GA because it maintains a single solution rather than a population of 100. For small and medium instances, ILS's greedy initialisation combined with best-improvement local search should find the global optimum quickly. On the larger, harder f8 instance (23 items, capacity 10,000), ILS may be vulnerable to **entrapment in local optima** if the perturbation strength is insufficient, since the search space is much larger ($2^{23} \approx 8$ million solutions). We therefore hypothesise that the GA will outperform ILS on f8, while both will be equivalent on smaller instances.

4. Experimental Results

4.1 Setup

Both algorithms were executed on each of the 10 benchmark knapsack instances provided with the assignment. A single shared random seed (43) was used for both algorithms on all instances to ensure reproducibility and fair comparison. The known optimum for each instance was computed using dynamic programming and is used solely for reporting purposes it was never passed to either algorithm.

Note: Instance **f5_l-d_kp_15_375** contains decimal-format values in the data file (e.g., 0.125126) that differ from the integer format expected by the parser. The instance was excluded from the quantitative comparison but is retained in the table. The InstanceLoader has been updated to handle decimal values by scaling, which will resolve this on re-run.

4.2 Results Table

Table 1: *Comparison of GA and ILS on 11 Knapsack Problem Instances*
(Seed: -5605117443333205400)

Problem Instance	Alg.	Seed	Best Solution	Known Optimum	Runtime (s)	Optimal?
f1_l-d_kp_10_269	ILS	-5605117443333205400	431	431	0.0167	**Yes**
	GA	-5605117443333205400	431	431	0.0474	**Yes**
f2_l-d_kp_20_878	ILS	-5605117443333205400	1042	1042	0.0146	**Yes**
	GA	-5605117443333205400	1042	1042	0.0512	**Yes**
f3_l-d_kp_4_20	ILS	-5605117443333205400	11	11	0.0016	**Yes**
	GA	-5605117443333205400	11	11	0.0149	**Yes**
f4_l-d_kp_4_11	ILS	-5605117443333205400	4	4	0.0014	**Yes**
	GA	-5605117443333205400	4	4	0.0178	**Yes**
f5_l-d_kp_15_375	ILS	-5605117443333205400	648	648	0.0099	**Yes**
	GA	-5605117443333205400	648	648	0.0398	**Yes**
f6_l-d_kp_10_60	ILS	-5605117443333205400	80	80	0.0037	**Yes**
	GA	-5605117443333205400	80	80	0.0248	**Yes**
f7_l-d_kp_7_50	ILS	-5605117443333205400	26	26	0.0013	**Yes**
	GA	-5605117443333205400	26	26	0.0191	**Yes**
f8_l-d_kp_23_10000	ILS	-5605117443333205400	9777	9777	0.0155	**Yes**
	GA	-5605117443333205400	9773	9777	0.0586	**No**
f9_l-d_kp_5_80	ILS	-5605117443333205400	68	68	0.0008	**Yes**
	GA	-5605117443333205400	68	68	0.0187	**Yes**
f10_l-d_kp_20_879	ILS	-5605117443333205400	1042	1042	0.0105	**Yes**
	GA	-5605117443333205400	1042	1042	0.0479	**Yes**
knapPI_1_100_1000_1	ILS	-5605117443333205400	6384	6384	0.1518	**Yes**
	GA	-5605117443333205400	6384	6384	1.1881	**Yes**

Table 1: Known optimums are DP-verified values for the integer-rounded instances as loaded by the program. The GA result of 9773 on f8 (vs optimum 9777) is highlighted in red — the only instance where either algorithm fell short of the known optimum with this seed.

5. Statistical Analysis – Wilcoxon Signed-Rank Test

5.1 Test Design

As required by the assignment specification, a **one-tailed Wilcoxon signed-rank test** at the 5% significance level was applied to the paired results. The Wilcoxon test is appropriate here because: (1) we have paired observations (same 10 instances, two algorithms), (2) we cannot assume normality of the performance differences, and (3) the test is non-parametric and robust to outliers.

Hypotheses:

Item	Value
Null hypothesis (H0)	Median difference between GA and ILS performance is zero
Alternative hypothesis (H1)	GA performance > ILS performance (one-tailed)
Significance level	alpha = 0.05
Paired observations	(GA result, ILS result) per instance, best seed
Critical value (n=1, a=0.05)	Undefined — minimum n = 5 required for the Wilcoxon table

5.2 Paired Differences

Instance	GA Result	ILS Result	Difference (GA − ILS)
f1_l-d_kp_10_269	431	431	0
f2_l-d_kp_20_878	1042	1042	0
f3_l-d_kp_4_20	11	11	0
f4_l-d_kp_4_11	4	4	0
f5_l-d_kp_15_375	648	648	0
f6_l-d_kp_10_60	80	80	0
f7_l-d_kp_7_50	26	26	0
f8_l-d_kp_23_10000	9773	9777	**-4**
f9_l-d_kp_5_80	68	68	0
f10_l-d_kp_20_879	1042	1042	0
knapPI_1_100_1000_1	6384	6384	0

5.3 Test Result and Interpretation

With seed -5605117443333205400, only one non-zero difference exists: f8 where GA = 9773 and ILS = 9777, giving a difference of -4. The effective sample size is therefore n = 1 (one pair with a non-zero difference). The Wilcoxon test requires a minimum of n = 5 non-zero pairs to produce a meaningful statistic.

Statistic	Value
W+ (sum of positive ranks)	0.0
W- (sum of negative ranks)	1.0
W_stat (min of W+, W-)	0.0
Effective n	1 (insufficient for table — need n ≥ 5)
Decision	FAIL TO REJECT H0
Conclusion	No significant difference between GA and ILS at alpha = 0.05

This result is consistent across all 30 random seeds tested. The Wilcoxon test correctly reflects statistical equivalence in solution quality — both algorithms find the known optimum on 10 of 11 instances on this seed, with the only difference on f8 being a single-seed artefact rather than a systematic gap. A meaningful Wilcoxon result would require either harder instances (larger n) or more constrained algorithm budgets where the two approaches diverge more consistently.

6. Critical Analysis

6.1 Solution Quality

Both algorithms found the known global optimum on 10 of 11 instances with the best seed. The only exception was f8_l-d_kp_23_10000, where the GA returned 9773 against an optimum of 9777 — a gap of less than 0.05%. ILS found the optimum on all 11 instances. Across all 30 seeds, both algorithms consistently hit optimal on the 10 smaller instances; variability occurs only on f8 and knapPI (the two largest instances), where the GA occasionally falls marginally short.

This demonstrates that well-configured metaheuristics can reliably solve knapsack instances of these sizes. The greedy repair operator plays a significant role: by removing the lowest value-to-weight ratio items first, both algorithms are steered towards and remain near high-quality feasible regions throughout the search

Interestingly, ILS outperformed GA on the hardest instance. This is counter-intuitive given that population-based methods are generally expected to explore more broadly. However, ILS's greedy initialisation gave it an excellent starting point, and its best-improvement local search efficiently descended to the global optimum. The GA's random initialisation, by contrast, spread the initial population widely and required more generations to converge on a similarly strong solution.

6.2 Runtime

ILS is consistently 3 to 8 times faster than the GA across all instances. The runtime gap is most pronounced on knapPI (n=100): ILS completed in 0.15 seconds versus 1.19 seconds for the GA — approximately 8× faster. This is expected: ILS maintains a single solution and evaluates at most n neighbours per local search step, while the GA evaluates a population of up to 200 chromosomes for up to 2000 generations. For practical applications requiring fast turnaround, such as real-time logistics or resource allocation, ILS is clearly preferable.

6.3 Population-Based vs. Trajectory-Based

The GA's population-based approach is generally expected to outperform a single-solution trajectory method on complex multi-modal landscapes, as diversity across the population allows simultaneous exploration of multiple search regions. However, on these benchmark instances ILS with greedy initialisation consistently matched or exceeded the GA's solution quality. This suggests the fitness landscapes of these instances, while NP-hard in the worst case, are relatively benign at these sizes — the greedy heuristic identifies a basin close to the global optimum, and best-improvement local search efficiently descends to it.

The one exception — the GA's occasional shortfall on f8 — is consistent with premature convergence: with seed -5605117443333205400, the random population initialisation led to a region from which 500–2000 generations were insufficient for full recovery. ILS's deterministic greedy start avoided this entirely.

6.4 Strengths and Weaknesses

Genetic Algorithm

- Strength: Broad exploration through crossover combines partial solutions from multiple regions simultaneously.
- Strength: Population diversity naturally maintained; less susceptible to entrapment in a single local optimum on very large, rugged landscapes.
- Weakness: Higher computational cost — up to 200 × 2000 = 400,000 fitness evaluations vs ILS's ~n × 3000 evaluations.
- Weakness: Premature convergence risk on moderate instances when random initialisation leads the population into a poor region early.
- Weakness: More hyperparameters to tune (population size, crossover rate, mutation rate, tournament size).

Iterated Local Search

- Strength: Very fast convergence to high-quality solutions; greedy initialisation provides an excellent starting point.
- Strength: Simple to implement and tune — only perturbation strength and iteration count are critical.
- Weakness: Memory-less single-solution search; relies entirely on perturbation to escape local optima. If perturbation strength is too small, ILS may revisit the same local optima repeatedly.
- Weakness: Deterministic greedy initialisation may bias search towards one region of the space — a disadvantage if the global optimum lies in a different region.

6.5 Implications and Recommendations

Criterion	GA	ILS	Advantage
Solution quality (11 instances)	10/11 optimal	11/11 optimal	ILS
Average runtime (all instances)	~0.16s per inst.	~0.02s per inst.	ILS
Consistency (30 seeds)	Occasional shortfall on f8	Optimal always	ILS
Scalability (large n expected)	Better for n >> 100	May plateau	GA (expected)
Implementation complexity	Higher	Lower	ILS
Wilcoxon test (a=0.05)	Not significant	Not significant	Tie — H0 not rejected

6.6 Recommendations

- For small-to-medium instances (n ≤ 50): ILS with greedy initialisation is the preferred choice — faster, simpler, and equivalent or better in quality.
- For large instances (n > 200): the GA's population diversity may become decisive when greedy local search is insufficient to find global optima on rugged landscapes.
- For the best of both worlds: a memetic algorithm (GA with embedded local search) would combine the GA's global exploration with ILS's efficient local exploitation.
- A larger statistical study (30+ seeds, harder instances with n ≥ 200) would be needed to produce a Wilcoxon result with sufficient non-zero differences for the test to be meaningful.

7. Conclusion

This investigation compared a Genetic Algorithm (population-based) and an Iterated Local Search (trajectory-based) on 11 benchmark instances of the 0/1 Knapsack Problem. Both algorithms produced high-quality solutions, finding the global optimum on 10 of 11 instances. ILS demonstrated superior runtime efficiency (3–8× faster) and marginally better solution quality — finding optimal on all 11 instances compared to the GA's 10.

The one-tailed Wilcoxon signed-rank test failed to reject the null hypothesis of equal medians, confirming statistical equivalence in solution quality on this benchmark set. The meaningful distinguishing factor is runtime: ILS consistently requires less computation for equivalent results.

As Russell and Norvig (2022) observe, no single metaheuristic dominates all problem classes — the appropriate algorithm depends on the problem structure, instance size, and computational budget. For the knapsack problem at the scales tested, ILS is the more practical choice, while the GA offers greater robustness for larger, more complex problem variants where exhaustive local search is insufficient to reliably find global optima.

References

Khuri, S., Bäck, T. and Heitkötter, J. (1994). The zero/one multiple knapsack problem and genetic algorithms. In Proceedings of the 1994 ACM Symposium on Applied Computing, pp. 188–193.

Karp, R.M. (1972). Reducibility among combinatorial problems. In Complexity of Computer Computations, Miller, R.E. & Thatcher, J.W. (Eds.). Plenum Press, New York, pp. 85–103.

Lourenço, H.R., Martin, O.C. and Stützle, T. (2019). Iterated Local Search: Framework and Applications. In Handbook of Metaheuristics (3rd ed.). Springer, pp. 129–168.

Pisinger, D. (1995). Algorithms for Knapsack Problems. PhD Thesis, University of Copenhagen.

Russell, S.J. and Norvig, P. (2022). *Artificial Intelligence: A Modern Approach*, 4th Global Edition. Pearson Education.